

“Fitness & Wellness Mobile App for Member Self-Service”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree
of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

E. PRANAY REDDY – 2520080061

B. DEEPAK – 2520080035

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology

[Insert KLH University logo here]

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

[Insert KLH University logo here]

Declaration

The Project Report entitled **“Fitness & Wellness Mobile App for Member Self-Service”** is a record of bonafide work of **E. PRANAY REDDY – 2520080061, B. DEEPAK – 2520080035** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 8, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

E. PRANAY REDDY – 2520080061

B. DEEPAK – 2520080035

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

[Insert KLH University logo here]

CERTIFICATE

This is to certify that the project report entitled **“Fitness & Wellness Mobile App for Member Self-Service”** is a bonafide work done and submitted by **E. PRANAY REDDY – 2520080061, B. DEEPAK – 2520080035** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 8, at KLH University during **2026–2027**, Trimester 4.

Signature of the Guide

Divya Priya Degala, Assistant Professor

Signature of the Course Coordinator

Sateesh Kumar

Signature of the HOD

Dr. Kaja Shareef

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or another in the completion of our project.

Our greatest appreciation goes to our Course Coordinator, Sateesh Kumar, and our guide, Divya Priya Degala, Assistant Professor, Department of Computer Science and Information Technology, for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to Dr. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

E. PRANAY REDDY – 2520080061

B. DEEPAK – 2520080035

ABSTRACT

The Fitness & Wellness Mobile App for Member Self-Service is a smart, user-friendly application designed to simplify fitness management and enhance the overall wellness experience of gym and fitness-centre members. Members traditionally depend on front-desk staff to register, renew memberships, book classes or trainers, and track their progress. This manual dependence wastes time, causes scheduling errors such as overbooked classes, and gives members little visibility into their own fitness journey.

The proposed system provides a mobile-first interface through which members can register, manage their membership, book fitness classes and trainer sessions, follow personalized exercise plans, log workouts, monitor body-measurement progress, and receive notifications and wellness guidance. Trainers can manage their classes and view enrolled members, while administrators can manage membership plans, members, trainers, schedules and reports from a central dashboard.

The application uses a React.js frontend, a Node.js and Express.js RESTful backend, and a MySQL relational database accessed through the Sequelize ORM. The database maintains relationships between users, membership plans, memberships, trainers, classes, bookings, workout plans, workout logs, progress records and notifications, while data integrity is enforced through primary keys, foreign keys, UNIQUE and CHECK constraints, transactions, views and stored logic. Authentication uses JSON Web Tokens (JWT) with role-based access control, and Twilio is integrated for SMS notifications such as booking confirmations and membership-expiry reminders.

By centralizing member services on one platform, the system promotes self-service, convenience, personalization and healthy lifestyle management, while helping fitness centres deliver efficient and modern customer service.

INDEX

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	9
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	10
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	11
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	12
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	14
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	16
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	18
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	20
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	21
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	22
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	23
13	References	Documentation, tools, and resources used	24
14	Appendices	Sample Code Snippets Installation / Setup Instructions Default Login Credentials API Reference Summary Screenshots	25

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Fitness centres, gyms and wellness studios serve a large number of members who each have different plans, schedules, goals and preferences. In many centres, day-to-day operations are still handled manually: members visit the front desk to join or renew a membership, call or message to book a class or a trainer, and rely on paper notebooks or memory to track their workouts. This works while the membership base is small but quickly breaks down as the centre grows, leading to overbooked classes, missed renewals, double-booked trainers, and members who lose motivation because they cannot see their own progress.

The Fitness & Wellness Mobile App for Member Self-Service was conceived to replace this manual coordination with a single, database-backed digital platform. Every member, membership, class, booking, workout and progress record is stored in one relational database, and the system itself – rather than a staff member cross-checking a register – is responsible for detecting expired memberships, full classes, duplicate bookings and clashing sessions.

1.2 Problem Statement

Given a fitness centre with a fixed set of membership plans, trainers and scheduled classes of limited capacity, and a continuous stream of members who want to join, renew, book sessions and track their health, the system must reliably answer the following questions in real time: whether a member currently holds a valid membership, whether a particular class or trainer session still has a free slot, whether the member is already booked into another session at the same time, and how the member is progressing toward their fitness goals. Answering these questions manually does not scale, is error-prone, and keeps members dependent on staff for even routine tasks.

1.3 Scope of the Project

The system covers the full self-service lifecycle of a fitness-centre member: registering an account; browsing and purchasing or renewing a membership plan; viewing the class and trainer schedule and booking or cancelling a slot; receiving a personalized workout plan and logging completed workouts; recording body measurements and viewing progress over time; and receiving notifications and wellness tips. Trainers and administrators are given management views for classes, members, plans and reports. Online payment gateway integration, wearable-device synchronization, and native Android/iOS packaging were treated as out of scope for this submission.

1.4 Objectives

- Allow members to register and manage their profile, membership and bookings independently, without front-desk assistance.
- Prevent classes from being overbooked and prevent a member or trainer from being double-booked, using a single, consistently applied time-overlap rule.

- Enforce membership rules: only members with an active, unexpired membership may book classes, and no member may book the same class twice.
- Provide personalized workout plans, workout logging and progress monitoring (weight, BMI and body measurements).
- Send timely SMS/in-app notifications for booking confirmations, class reminders and membership expiry.
- Give trainers and administrators dashboards to manage classes, members, plans and reports.
- Secure the system with JWT authentication and role-based access control so that members can never reach trainer- or admin-only functionality.

1.5 Importance of the Application

Self-service reduces the workload of front-desk staff, saves members' time, and gives them access to essential services anytime and anywhere. For the fitness centre, a centralized system provides accurate real-time information on class occupancy, membership renewals and member engagement, enabling better planning. For members, visible progress and timely reminders improve motivation and consistency, which is the single most important factor in achieving fitness goals.

1.6 Target Users / Audience

The system serves three roles. Members use it to register, manage memberships, book classes and trainers, follow workout plans, log workouts, track progress and read wellness guidance. Trainers use it to manage their class schedule, view enrolled members and assign workout plans. Administrators use it to manage membership plans, members, trainers, classes, notifications and reports.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has modest hardware requirements, since both the Node.js backend and the MySQL database are lightweight relative to the dataset generated by a single fitness centre. A development or demonstration machine with a dual-core processor, 4 GB of RAM and roughly 1 GB of free disk space (for Node modules, MySQL data files and project source) is sufficient to run the entire stack – frontend, backend and database – on one computer. Members only need a smartphone or computer with a modern browser and an internet connection.

2.2 Software Requirements

- Node.js 18 or later, with npm
- MySQL 8.0 or later
- A modern web browser (Chrome, Edge or Firefox; mobile Chrome/Safari for the member view)
- Git, for version control
- Postman, for API testing
- A Twilio account (trial is sufficient) for SMS notifications
- MySQL Workbench and Visual Studio Code for development (optional but recommended)
- Docker (optional), for containerized deployment

2.3 Development Tools and Frameworks

React.js (bootstrapped with Vite) is used for the frontend; Node.js with the Express.js framework for the backend REST API; MySQL as the relational database, accessed through the Sequelize ORM with the mysql2 driver; JSON Web Tokens for authentication; Twilio for SMS notification services; Postman for API testing and documentation; and Git with GitHub for source control. Visual Studio Code is the primary development environment, and MySQL Workbench is used for inspecting and verifying the database schema and data directly.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a mobile-first single-page application built with React.js and bundled using Vite. React Router handles client-side navigation between the public (login/register), member, trainer and admin areas, and Axios is used as the single HTTP client for every API call, wrapped in one central service module per backend resource (memberships, classes, bookings, workouts, progress, notifications). Styling uses plain CSS with a small set of reusable design tokens (colours, spacing, cards, badges) and a responsive layout so that the interface behaves like a mobile app on small screens.

3.2 Backend

The backend is a RESTful API built with Node.js and Express.js, organized into routes, controllers, services and models: routes declare the URL and required role, controllers hold the request/response and validation logic, services hold multi-step business rules (such as class booking), and Sequelize models describe the tables and their associations. This separation keeps each file focused on one responsibility.

3.3 Database

MySQL 8 stores all persistent data across ten related tables (users, membership_plans, memberships, trainers, classes, class_bookings, workout_plans, workout_logs, progress_records and notifications). Referential integrity is enforced with foreign keys, uniqueness with UNIQUE constraints (for example one email per user, one booking per member per class), and data validity with CHECK constraints (for example positive class capacity, end time after start time). Views and stored logic are used for recurring reporting and maintenance tasks, and multi-table operations are wrapped in transactions. The Sequelize ORM maps tables to JavaScript models and is used together with raw SQL where a query is better expressed directly.

3.4 Version Control

Git is used throughout development for version control, with the project structured as a single repository containing both the client and server codebases, hosted on GitHub. Both team members work on feature branches that are merged into the main branch after review.

3.5 Authentication and Security Libraries

Passwords are hashed with bcrypt before being stored – plain-text passwords are never written to the database. Login sessions are stateless JSON Web Tokens (JWT), signed with a server-side secret and carrying only the user's ID, role and name. Express middleware verifies this token on every protected route and enforces role-based access, ensuring that member accounts cannot reach trainer- or admin-only endpoints even if a request is crafted by hand. Twilio is

used as the SMS gateway for booking confirmations and reminders; its credentials are held only in environment variables.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a standard three-tier architecture, cleanly separating the user interface, the business logic and the persisted data. Figure 4.1 shows the three layers and how a request flows between them.

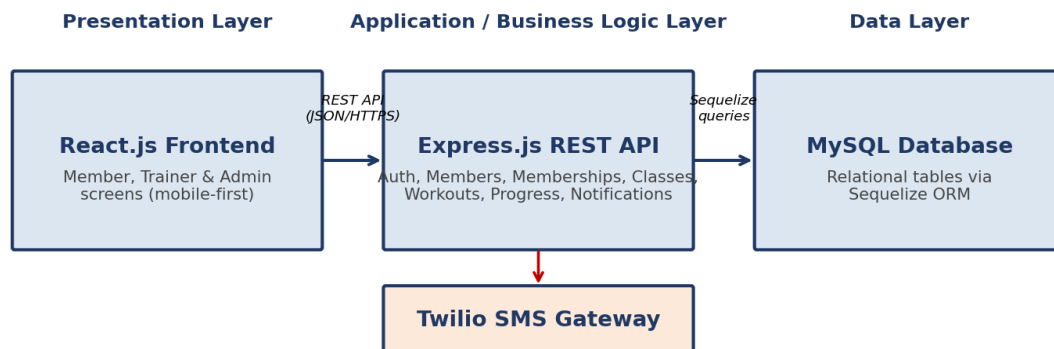


Figure 4.1: High-level three-tier architecture of the Fitness & Wellness Mobile App

4.2 Description of Each Layer

The Presentation Layer is the React single-page application. It never talks to MySQL directly; every piece of data it shows comes from calling the REST API over HTTP. During development, Vite’s built-in proxy forwards any request the frontend makes to “/api/...” straight through to the Express server, so the browser never needs to know the backend’s actual port or address.

The Application Layer is the Express.js REST API. Incoming requests pass through authentication middleware (which verifies the JWT and attaches the requesting user’s identity and role), then into a controller that validates the request and applies business rules – for example, checking that a member’s membership is active, that the class still has a free seat and that the member has no overlapping booking – before reading or writing data through a Sequelize model. Notification events are handed to a Twilio service module that sends the SMS and records an in-app notification.

The Data Layer is the MySQL database, accessed through a Sequelize connection pool rather than a single long-lived connection, so multiple requests can be served concurrently without waiting on each other unnecessarily.

4.3 Deployment Architecture

For development and demonstration purposes, all three layers run as separate local processes on a single machine: the Express server listens on port 5000, the Vite development server serves the React app on port 5173 and proxies API calls to it, and MySQL listens on its default port 3306. A Docker Compose configuration (one container each for the client, server and database)

is provided as an optional way to start the whole stack with a single command. Chapter 11 outlines how the system could be extended to a hosted cloud environment.

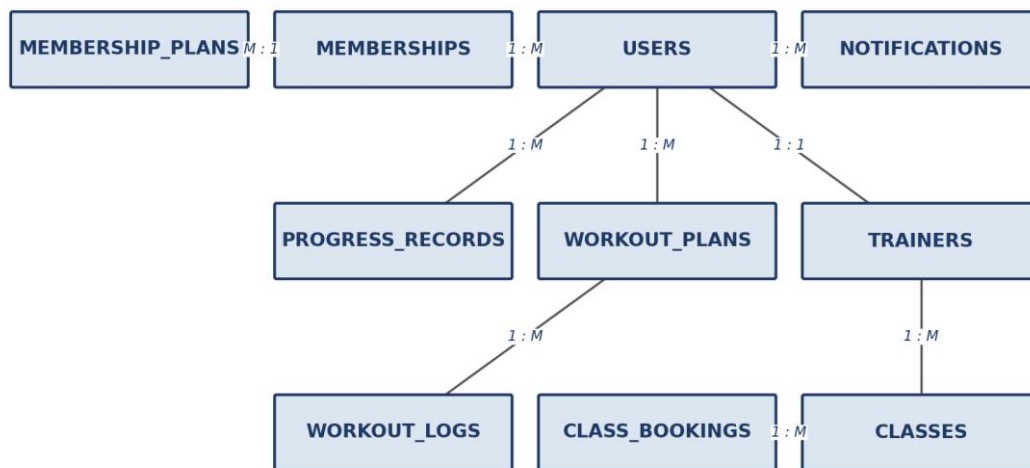
CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema is normalized (to Third Normal Form) to remove redundant data: each fact is stored in exactly one place, and every relationship between entities is expressed through a foreign key rather than by duplicating data across tables. Every table enforces its own invariants directly in the database – not just in application code – using PRIMARY KEY, FOREIGN KEY, UNIQUE and CHECK constraints, so that invalid data (a class with negative capacity, an end time before a start time, a duplicate booking) is rejected at the database level even if a bug elsewhere in the application tried to insert it.

5.2 ER Diagram / Database Schema

The database consists of ten related tables; the main relationships are shown in Figure 5.1.



CLASS_BOOKINGS, WORKOUT_LOGS and MEMBERSHIPS also carry a user_id foreign key to USERS (not drawn, to keep the figure readable).

Figure 5.1: Entity-Relationship diagram of the database schema

Table	Purpose and key columns
USERS	Every account (member, trainer or admin): user_id, name, email (UNIQUE), phone, password_hash, role, date_of_birth, gender, height_cm, status.
MEMBERSHIP_PLANS	Plans offered by the centre: plan_id, plan_name, duration_days, price, features, is_active.
MEMBERSHIPS	A member's subscription to a plan: membership_id, user_id (FK), plan_id (FK), start_date, end_date, status (Active / Expired / Cancelled).
TRAINERS	Trainer profile linked 1:1 to a user: trainer_id, user_id (FK), specialization, experience_years, bio.
CLASSES	Scheduled classes / sessions: class_id, class_name, trainer_id (FK), start_time, end_time, capacity (CHECK > 0), room/location, status.

Table	Purpose and key columns
CLASS_BOOKINGS	A member's booking of a class: booking_id, class_id (FK), user_id (FK), booked_at, status; UNIQUE(class_id, user_id).
WORKOUT_PLANS	Personalized exercise plans: plan_id, user_id (FK), trainer_id (FK), title, goal, exercises (JSON), start_date, end_date.
WORKOUT_LOGS	Completed workouts: log_id, user_id (FK), workout_plan_id (FK), log_date, duration_min, calories_burned, notes.
PROGRESS_RECORDS	Body measurements over time: record_id, user_id (FK), record_date, weight_kg, body_fat_pct, chest/waist/hip cm, bmi.
NOTIFICATIONS	In-app and SMS messages: notification_id, user_id (FK), type, message, channel, is_read, created_at.

Two database objects support reporting and maintenance: a view (vw_class_occupancy) that joins classes with the count of confirmed bookings so that seats-left can be read directly, and a stored procedure (sp_expire_memberships) that marks memberships whose end_date has passed as Expired and queues expiry notifications.

5.3 Design Decisions and Assumptions

- **Single source of truth for time conflicts:** all overlap checks for a member's bookings and for a trainer's classes read from the CLASSES and CLASS_BOOKINGS tables using one overlap rule, so there is no second table that could drift out of sync.
- **Membership validity is derived:** a member is considered "active" only when a membership row exists with status Active and end_date on or after today; the nightly stored procedure keeps the status column consistent with the dates.
- **Roles are fixed at account creation:** public registration always creates a Member. Trainer and Admin accounts are created by an administrator.
- **Payments are out of scope:** purchasing a plan records the subscription directly; a payment-gateway integration is listed under future enhancements.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The backend is organized into eight resource modules, each with its own routes, controller and model file: Authentication, Members, Membership Plans & Memberships, Trainers, Classes, Bookings, Workouts & Progress, and Notifications.

The most involved piece of logic is the class-booking service. When a member requests a booking, it runs the following checks in sequence – the class exists and is open for booking; the member holds an active, unexpired membership; the member has not already booked this class; the member has no other booking that overlaps in time; and the class still has a free seat – and only if every check passes does it insert the booking row and an in-app notification, inside a single database transaction. The class row is read with a row lock (`SELECT ... FOR UPDATE`) so that two members booking the last seat at the same instant cannot both succeed. A failure partway through rolls back and leaves no partial state.

The core conflict-detection rule used throughout the system is: a requested time slot conflicts with an existing one if $(\text{existing_start} < \text{requested_end}) \text{ AND } (\text{existing_end} > \text{requested_start})$. This single rule is applied consistently when a member books a class, when a trainer is assigned to a class, and when an administrator creates or reschedules a class, so back-to-back sessions are allowed but genuine overlaps never are.

6.2 Front-end Logic

The frontend uses React's Context API for the two pieces of state that genuinely need to be shared across the whole application – the logged-in user (AuthContext) and toast notifications (ToastContext) – and otherwise keeps state local to the component that needs it. This avoids the added complexity of a state-management library such as Redux, which is unnecessary at this project's scale. Shared UI elements (a Modal, a ConfirmDialog, status badges, progress charts and loading/empty/error placeholders) are each written once and reused across the member, trainer and admin pages, so every page loads, fails and confirms actions in a visually consistent way.

6.3 API Integration

All HTTP calls go through one central Axios instance, which automatically attaches the logged-in user's JWT to every outgoing request and automatically logs the user out if any response comes back with a 401 Unauthorized status (for example, after the token expires). Each backend resource has a matching frontend service file (`membershipService.js`, `classService.js`, `bookingService.js`, and so on) so that no page ever hardcodes a URL or repeats the same fetch logic.

6.4 Authentication and Authorization

Registration always creates a Member account – there is deliberately no way to create a trainer or admin account through the public API, so a curious member cannot register their way into elevated access. Every protected route runs a `protect` middleware that verifies the JWT, and role-restricted routes additionally run a `restrictTo(role)` middleware that checks the token's role claim, rejecting any unauthorized token with a 403 Forbidden response. Passwords are hashed with `bcrypt`, and login failures return a generic message that does not reveal whether the email or the password was wrong.

CHAPTER 7: FEATURES

7.1 List of Main Features

Member-facing features:

- Self-registration, login and profile management
- Browse membership plans; subscribe to, renew or cancel a membership
- View the class timetable (filterable by type, trainer and day) with live seats-left
- Book and cancel classes and trainer sessions, with duplicate and overlap protection
- Personalized workout plans assigned by trainers; workout logging
- Progress monitoring: weight, BMI and body measurements with history charts
- In-app notifications and SMS alerts; wellness tips and guidance

Trainer-facing features:

- Create and manage own classes; view enrolled members per class
- Create and assign workout plans to members; view member progress

Administrator-facing features:

- Manage membership plans, members, trainers and classes
- Dashboard: active members, expiring memberships, today's classes, class occupancy and new sign-ups
- Send announcements and reminders by SMS / in-app notification

7.2 How Each Feature Works

- **Class booking:** when a member taps Book, the server verifies the member's active membership, checks for duplicates and overlaps, locks the class row, confirms a seat is free, then inserts the booking and a notification in one transaction. If the class is full, the member is told so and may join a waiting list in a future version.
- **Membership management:** subscribing creates a membership row with `start_date = today` and `end_date = start_date + plan duration`. Renewing before expiry extends from the current `end_date`; renewing after expiry starts from today. A scheduled job runs `sp_expire_memberships` daily and sends a reminder SMS seven days before expiry.
- **Progress and BMI:** each time a member records their weight, the system computes $BMI = \text{weight(kg)} \div \text{height(m)}^2$ and classifies it as Underweight (below 18.5), Normal (18.5–24.9), Overweight (25–29.9) or Obese (30 and above). The progress page plots weight and BMI over time and shows the percentage of the member's goal achieved.
- **Workout plans and logging:** a trainer assigns a plan (title, goal and a list of exercises with sets and repetitions). Members mark workouts as completed and log duration and calories; weekly totals are shown on the dashboard to encourage consistency.
- **Notifications:** events such as booking confirmation, class cancellation and membership expiry create an in-app notification and, if the member has a phone number, an SMS

through Twilio. A failure in the SMS gateway never blocks the underlying booking; it is logged and retried.

- **Role-based dashboards:** after login, the JWT role decides which area of the application is shown (member, trainer or admin), and the same role is re-checked on the server for every request.

CHAPTER 8: TESTING

8.1 Postman API Testing

A Postman collection is built covering every REST endpoint, organized into folders by resource (Auth, Members, Plans & Memberships, Trainers, Classes, Bookings, Workouts & Progress, Notifications) plus a Health check. The Login requests carry a small test script that automatically saves the returned JWT into a collection variable, and requests such as Create Class, Book Class and Log Workout are chained together using those variables, so a full workflow can be exercised without manually copying any ID or token between requests.

8.2 Integration Testing

Beyond Postman, each backend module is tested against a real, running MySQL database at every stage of development – each module is verified with the actual database before the next module is built on top of it, rather than being tested in isolation with mocked data. As a final integration check, the entire database is dropped and rebuilt from schema.sql and seed.sql from zero, and the full test checklist below is re-run against that fresh install to confirm that the system works for a first-time setup.

8.3 Tools Used for Testing

- Postman – manual and scripted API testing
- curl (command line) – scripted regression checks across development phases
- MySQL command-line client / Workbench – direct verification of table contents and constraint behaviour
- Node.js testing tools (Jest and Supertest) – automated unit and API tests for the booking and membership logic
- Manual browser testing (desktop and mobile viewport) – verifying the React frontend’s behaviour and appearance

8.4 Test Cases and Results

The test cases below are planned for the system. The “Actual Result” column is to be filled in with Pass / Fail after the tests are executed on the final build.

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	401, login rejected	To be updated
2	Member token calls an admin-only route	403 Forbidden	To be updated
3	Register with an email that already exists	409 Conflict, friendly message	To be updated
4	Book a class without an active membership	403, “active membership required”	To be updated

#	Test Case	Expected Result	Actual Result
5	Book the same class twice	Second attempt returns 409 Conflict	To be updated
6	Book a class that is full	409, "class is full"	To be updated
7	Book a class overlapping another booked class	409, message naming the clashing class	To be updated
8	Book a class that starts exactly when another ends	No conflict (back-to-back allowed)	To be updated
9	Two members book the last seat simultaneously	Exactly one succeeds; capacity never exceeded	To be updated
10	Create a class with capacity ≤ 0 or end before start	400, rejected before reaching the database	To be updated
11	Assign a trainer to two overlapping classes	409, trainer already scheduled	To be updated
12	Renew an expired membership	New end_date calculated from today	To be updated
13	Run sp_expire_memberships	Past-due memberships set to Expired	To be updated
14	Record weight and height for BMI	BMI and category computed correctly	To be updated
15	Twilio failure during booking	Booking still succeeds; error logged	To be updated

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

- Load the schema and seed data: `mysql -u root -p < database/schema.sql`, then ... `< database/seed.sql`
- Start the backend: `cd server, npm install, npm run dev` (listens on port 5000)
- Start the frontend: `cd client, npm install, npm run dev` (listens on port 5173)
- Open `http://localhost:5173` in a browser (or on a phone connected to the same network) and log in with a seeded account
- Optional: run `docker compose up` to start the client, server and database containers together

9.2 Environment Configuration

All server configuration is kept in a single `.env` file (never committed to version control), covering the database connection (host, port, user, password, database name), the JWT signing secret and token expiry, the port the server listens on, the frontend's URL for CORS, and the Twilio credentials (account SID, auth token and sender phone number). A `.env.example` template with placeholder values is committed instead, so the required variables are documented without exposing any real credentials.

9.3 Hosting of Frontend and Backend

For this submission, the frontend and backend run as local Node.js processes (optionally in Docker containers) and the database runs as a local MySQL instance, which is appropriate for development and in-class demonstration. A production deployment was not required for this course submission; Chapter 11 outlines how the same codebase can be adapted to a hosted environment (for example, the React build on a static host, the Express API on a cloud VM or container service, and a managed MySQL instance).

CHAPTER 10: CHALLENGES & LIMITATIONS

Note: this chapter should be updated by the team with the actual issues faced during development. The items below are the main design challenges identified for this project and the approach taken for each.

10.1 Issues Faced During Development

- **Overbooking under concurrent requests:** two members booking the last seat of a class at nearly the same time could both pass a simple “seats left?” check.
- **Keeping membership status accurate:** a status column can become stale when a membership passes its end date without any request touching it.
- **Time-zone handling:** class times stored in UTC must be displayed in the member’s local time and pre-filled correctly in edit forms.
- **Third-party SMS reliability:** an SMS gateway outage or invalid number must not make a booking fail.

10.2 Solutions Applied

- Class booking runs inside a transaction that locks the class row (SELECT ... FOR UPDATE) before counting confirmed bookings, so only one concurrent request can take the last seat.
- A stored procedure, `sp_expire_memberships`, is run daily and membership validity is also checked by date at booking time, so correctness never depends solely on the scheduled job.
- A shared `dateUtils` helper converts ISO timestamps to the browser’s local time using local-time `Date` getters rather than raw string slicing or UTC getters.
- Notification sending is decoupled from the booking transaction: SMS errors are caught, logged and retried, and the in-app notification is always stored.

10.3 Current Limitations or Known Bugs

- No online payment gateway; membership purchase is recorded without taking payment.
- No email verification or password-reset flow currently exists for user accounts.
- Trainer and admin accounts can only be created by an administrator or by inserting a database row; there is no public sign-up path to these roles, by design.
- No waiting list for full classes and no wearable-device integration.
- The system has been tested on a single local deployment and has not yet been deployed to a public cloud environment. The interface is a responsive web app rather than a packaged native mobile application.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- Online payments for membership purchase and renewal (for example Razorpay or Stripe).
- Email-based password reset and account verification.
- Waiting lists for full classes, with automatic promotion when a seat frees up.
- A calendar view of the class timetable and of a member's upcoming bookings.
- QR-code based check-in at the centre and for classes, to track real attendance.
- Diet and nutrition tracking, and AI-assisted personalized workout recommendations.

11.2 Possible Integrations or Optimizations

- Packaging the app as a Progressive Web App (PWA) or with React Native / Capacitor for installable Android and iOS apps with push notifications.
- Synchronizing steps, heart rate and sleep from wearable devices and health platforms.
- Caching frequently requested timetable queries to reduce database load during peak booking hours.
- Deploying with Docker on a cloud platform with a CI/CD pipeline and a managed MySQL service.
- Expanding the automated test suite (Jest and Supertest) so that regressions are caught automatically on every change.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project sets out to replace manual, front-desk-dependent fitness-centre operations with a single, database-driven self-service application. A normalized ten-table MySQL schema, a RESTful Express API with Sequelize models, and a React frontend for members, trainers and administrators were designed so that each layer can be built and verified against a real, running database before the next is added.

12.2 What Was Achieved

The system provides role-based authentication; membership management; class and trainer-session booking that prevents overbooking, duplicate bookings and time overlaps through one consistently applied rule; personalized workout plans and workout logging; progress and BMI monitoring; and SMS/in-app notifications. Administrators and trainers are given dashboards for managing plans, members, classes and reports, and the testing process includes a full fresh-database rebuild and a systematic checklist.

12.3 Skills Learned

Working on this project gave practical experience in relational database design and normalization; enforcing constraints, views and stored logic at the database layer rather than trusting application code alone; using an ORM (Sequelize) alongside raw SQL; designing a REST API with consistent response conventions and meaningful HTTP status codes; managing multi-table transactions and row-level locking to keep related data consistent under concurrency; integrating a third-party service (Twilio); building a React frontend with Context-based state management; and systematic testing, including deliberately trying to break the system's own rules to confirm they hold.

CHAPTER 13: REFERENCES

- React – Official Documentation, <https://react.dev>
- Vite – Official Documentation, <https://vitejs.dev>
- Node.js – Official Documentation, <https://nodejs.org/en/docs>
- Express.js – Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- Sequelize – Official Documentation, <https://sequelize.org>
- mysql2 (Node.js MySQL driver) – npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken – npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt – npm documentation, <https://www.npmjs.com/package/bcrypt>
- Twilio – Programmable Messaging Documentation, <https://www.twilio.com/docs/messaging>
- Docker – Official Documentation, <https://docs.docker.com>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs – JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The core time-overlap query, used consistently across the system (here, checking whether a member already has a booking that clashes with a requested class):

```
-- The single overlap rule used wherever a schedule is checked
SELECT c.class_id, c.class_name, c.start_time, c.end_time
FROM class_bookings b
JOIN classes c ON b.class_id = c.class_id
WHERE b.user_id = ?
      AND b.status = 'Confirmed'
      AND c.start_time < ?    -- requested end time
      AND c.end_time   > ?;  -- requested start time
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cp server/.env.example server/.env` and fill in database, JWT and Twilio values
- `cd server && npm install && npm run dev` (backend, port 5000)
- `cd client && npm install && npm run dev` (frontend, port 5173)
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

All seeded accounts use the password “password123”. (Update this table to match your actual seed.sql before submission.)

Role	Email	Notes
Admin	admin@fitwell.com	Full access
Trainer	trainer1@fitwell.com	Manages classes and workout plans
Member	pranay@fitwell.com	Sample member
Member	deepak@fitwell.com	Sample member

Appendix D: API Reference Summary

All endpoints are prefixed with `/api/<resource>`. Full request/response examples are in the accompanying Postman collection.

Resource	Endpoints
Auth	POST /register, POST /login, GET /me
Members	GET /, GET /:id, PUT /:id, DELETE /:id
Plans	GET /, POST /, PUT /:id, DELETE /:id
Memberships	POST /subscribe, GET /my, PUT /:id/renew, PUT /:id/cancel

Resource	Endpoints
Classes	GET /, GET /:id, POST /, PUT /:id, DELETE /:id
Bookings	POST /, GET /my, GET /, DELETE /:id
Workouts	GET /plans/my, POST /plans, PUT /plans/:id, POST /logs, GET /logs/my
Progress	POST /, GET /my
Notifications	GET /my, PUT /:id/read, POST /send



The login form is titled 'Login to your account' and includes a 'Welcome back' message. It prompts the user to enter their email and password. The email field has a placeholder 'e.g. member1@example.com'. The password field has a placeholder 'Enter your password'. A 'Login' button is provided. Below the login button, there are 'Quick fill credentials' for 'Member', 'Trainer', and 'Admin'. At the bottom, there are links for 'Need an account? Register' and 'Back to Home'.